CAMPUS RECRUITMENT AUTOMATION ECOSYSTEM

Smart Campus Placement Management Platform

End-to-End Working Flow, State Machine Architecture & Technical Interview Master Guide

PART 1: SYSTEM ARCHITECTURE & WORKING FLOW

1. High-Level Technology Architecture

The platform is an enterprise full-stack recruitment automation solution that replaces manual spreadsheets and uncoordinated circulars with a centralized real-time hiring engine.

Architecture Layers:

- Frontend: React 19 + Vite 8.2 (Context API, Custom Hooks, Modern CSS Custom Properties)
- Backend API: Spring Boot 3.4.3 & Java 23 (REST Controllers, Service State Machine, Jakarta Bean Validation)
- Persistence & Security: MySQL 8.0 with Spring Data JPA / Hibernate ORM, BCrypt Password Encryption

2. Recruitment Workflow State Machine

Every student application progresses through a strict, deterministic state machine enforced by ApplicationService and InterviewService:

```
[APPLIED] % % %º [SHORTLISTED] % % %º [INTERVIEW] % % %º [SELECTED] % % %º [OFFER GENERATED] % % %º [ACCEP
```

- **Stage 1 (Applied):** Student submits application after passing the 4-point Pre-Eligibility Rule Check.
- **Stage 2 (Shortlisted):** Placement Officer (Admin) or Recruiter evaluates credentials and shortlists valid candidates.
- **Stage 3 (Interview Scheduled):** Recruiter books technical or HR round with date, time, mode (virtual/offline), and panel name. Instant notification dispatches to the student.
- **Stage 4 (Interview Outcome):** Marking interview PASSED automatically transitions application to SELECTED. Marking FAILED transitions directly to REJECTED.
- **Stage 5 (Formal Offer Released):** Recruiter or TPO rolls out official placement offer with annual CTC (LPA) and joining date.
- **Stage 6 (Candidate Decision):** Student reviews terms and formally clicks ACCEPT (permanently registering placement in campus analytics) or DECLINE.

3. User Roles & Capabilities

- **Ø<ß“ Student Portal:** Profile & resume management, live pre-eligibility evaluation on job cards, 1-click application submission, real-time stage badges, notification center alerts, and offer acceptance.
- **Ø<ßÛþ College Management (Admin):** Executive Placement Intelligence dashboard (placement % rate, department distribution, avg/highest LPA), talent pool inspection, partner employer registration, campus job drive publishing, and candidate pipeline tracking.
- **Ø=Ü¼ Campus Recruiter Portal:** Company-specific applicant screening, CV review, technical/HR interview scheduling, Pass/Fail result logging, and formal offer letter release.

4. Deterministic Pre-Eligibility Rule Engine

Encapsulated in EligibilityService.java, this engine evaluates four deterministic criteria before any application can be created:

Rule	Evaluation Logic	User Explanation On Failure
1. Academic Cutoff	Student.CGPA >= Job.MinCGPA	CGPA is below the minimum requirement
2. Branch Match	Job.Branch == "ALL" || Allowed.contains(Branch)	Department is not eligible for this opening
3. Backlog Limit	Student.Backlogs <= Job.MaxBacklogs	Active backlogs exceed maximum allowed limit
4. Application Deadline	LocalDate.now() <= Job.Deadline	Application deadline for this drive has passed

PART 2: HIGH-IMPACT TECHNICAL & BEHAVIORAL INTERVIEW Q&As

[OVERVIEW] Q1: Can you give a 2-minute elevator pitch of your Smart Campus Placement project?

Smart Campus Placement is an enterprise-grade recruitment platform designed to replace manual spreadsheets, delayed circulars, and fragmented emails in college placements.

It connects three key stakeholders—Students, Placement Officers (TPOs), and Corporate Recruiters—in a unified automated ecosystem. The core innovation is our real-time eligibility rule engine that pre-evaluates student academic metrics (CGPA cutoff, engineering branch matching, backlog limits, and deadlines) before applications are submitted, alongside an automated recruitment state machine that manages the pipeline from shortlisting and interview scheduling to formal offer release and visual placement analytics.

[TECH STACK] Q2: What is the technology stack used, and what was your design rationale?

- **Spring Boot 3.4.3 & Java 23:** Chosen for robust enterprise dependency injection, declarative transactions (@Transactional), type safety, and Jakarta Bean Validation.
- **MySQL 8.0 & Hibernate ORM:** Recruitment processes demand strict relational integrity, composite unique constraints, and ACID guarantees across applications, interviews, and offers.
- **Spring Security Crypto (BCrypt):** Implements salted, multi-round adaptive password hashing with zero plain-text storage.
- **React 19 + Vite 8.2:** Provides sub-second Hot Module Replacement (HMR), Context API for global session state, and modular single-page UI.
- **Modern Vanilla CSS:** Handcrafted CSS Custom Properties (tokens) and glassmorphic aesthetics for full flexibility without third-party CSS overhead.

[BACKEND CORE] Q3: How does the Eligibility Rule Engine work in the backend?

The eligibility evaluation is encapsulated in EligibilityService.java. When a student views an opening or submits an application, the engine executes four deterministic validation rules:

1. Academic Cutoff: student.getCgpa() >= job.getMinCgpa()
2. Branch Match: job.getAllowedBranch().equals("ALL") || allowedBranches.contains(student.getBranch())
3. Backlog Constraint: student.getBacklogs() <= job.getMaxBacklogs()
4. Deadline Enforcement: LocalDate.now().isBefore(job.getDeadline()) || isEqual()

We also exposed a dedicated pre-check endpoint (GET /applications/check-eligibility). If any rule fails, the service returns an EligibilityResult with isEligible: false and specific human-readable reasons so candidates immediately know why they are ineligible.

[PIPELINE] Q4: How is the recruitment state machine governed? What happens if an interview passes or fails?

The recruitment lifecycle is modeled as an automated state machine across ApplicationService.java and InterviewService.java:

- **Application Creation:** Applications initialize in APPLIED status upon submission.
- **Shortlisting:** Only applications in APPLIED state can be transitioned to SHORTLISTED.
- **Interview Scheduling:** Interviews can only be booked for candidates currently in SHORTLISTED state.
- **Automatic State Transition:** When an interviewer submits a result, marking PASSED automatically transitions the application to SELECTED and notifies the candidate. Marking FAILED transitions it to REJECTED.
- **Offer Generation:** Formal offers (specifying CTC and reporting date) can only be generated for candidates in SELECTED status.

[CONCURRENCY] Q5: How do you prevent duplicate applications from the same student for the same job?

We implement defense-in-depth across two distinct architectural layers:

- **1. Service Layer Pre-Check:** In ApplicationService.applyForJob(), we query applicationRepository.findByStudentIdAndJobId(studentId, jobId). If found, an IllegalArgumentException is thrown before any write happens.
- **2. Database Unique Constraint:** On the MySQL table application, we enforce a composite unique constraint: @UniqueConstraint(name = "uk_student_job", columnNames = {"student_id", "job_id"}). Even under high-concurrency race conditions, the database guarantees referential uniqueness.

[SECURITY] Q6: How does authentication and Role-Based Access Control (RBAC) work in your platform?

Authentication is strictly database-driven via AuthService.java:

- **Password Security:** Passwords are encrypted using BCrypt (10 salt rounds).
- **Database Validation:** Upon login, the system queries studentRepository.findByEmail(email), verifies password matching via passwordEncoder.matches(), and extracts the stored role (STUDENT, ADMIN, or RECRUITER).
- **Zero Hardcoded Bypasses:** All demo bypasses and mock user IDs were removed. All users must exist in the database with valid hashed passwords.
- **Frontend Role Routing:** The React AuthContext stores the session and dynamically routes users: students see the Job Portal, recruiters see the Candidate Evaluation Console, and placement officers see the Executive Intelligence Dashboard.

[DATABASE] Q7: Can you describe your database entity relationships?

The relational schema consists of 7 normalized JPA entities:

- **Student:** User entity with authentication credentials, academic metrics (CGPA, backlogs, branch, graduation year), resume link, and role.
- **Company & Job:** Company stores corporate partner details. Job has a ManyToOne link to Company and stores hiring criteria.
- **Application:** Relational join entity connecting Student (ManyToOne) and Job (ManyToOne) with recruitment stage status.
- **Interview & Offer:** Both entities have ManyToOne links to Application, recording round logistics/outcomes and compensation CTC/acceptance.
- **Notification:** ManyToOne link to Student, storing system alerts with read/unread tracking.

[FRONTEND STATE] Q8: How did you manage global state and data synchronization in React without Redux?

We utilized the React Context API combined with a centralized custom hook:

- **AuthContext:** Manages user session, role resolution, login, registration, and logout with persistence in localStorage.
- **ToastContext:** Provides non-blocking, auto-dismissing animated feedback notifications across components.
- **usePlacementData Hook:** Centralizes data fetching across jobs, companies, applications, interviews, offers, and student records using parallel Promise.all calls. It exposes a synchronized refreshData() callback that modals invoke after mutations.

[CHALLENGES] Q9: What was a major technical or security challenge you resolved in this project?

A major challenge was hardening the authentication architecture from a prototyping setup into production-ready security.

Initially, the platform contained hardcoded demo bypasses for admin and recruiter logins with dummy IDs (0L, 999L) and fallback passwords. This bypassed database checks and made it impossible to audit real user actions.

I refactored AuthService.java to eliminate all bypass blocks, enforced strict database lookups with BCrypt encryption across all roles, purged test accounts from MySQL while preserving real student application history, and updated the UI into a single unified role-detecting login modal. All 32 automated unit and integration tests passed cleanly.

[SCALABILITY] Q10: How would you scale this platform to support 100,000 students during placement season?

To support massive concurrent traffic spikes:

- **Database Optimization:** Add composite indexes on application(student_id, status) and job(status, deadline). Implement MySQL Read Replicas for read-heavy job browsing.
- **In-Memory Caching (Redis):** Cache active company listings (/companies) and open campus jobs (/jobs), offloading 80% of database read traffic.
- **Asynchronous Queues (RabbitMQ / Kafka):** Offload email dispatches, candidate notifications, and interview schedule alerts to background workers.
- **Stateless JWT Tokens:** Replace stateful session tokens with cryptographically signed JSON Web Tokens (JWT) to allow horizontal scaling behind an AWS ALB or NGINX load balancer.

PART 3: HIGH-YIELD INTERVIEW BUZZWORD CHEAT SHEET

Domain	Key Concepts & Keywords to Speak During Interview
Backend Architecture	Spring Boot 3.4.3, Java 23, RESTful APIs, Jakarta Validation, Dependency Injection
Data Persistence	Spring Data JPA, Hibernate ORM, MySQL 8.0, ACID, Composite Unique Constraints
Recruitment Core	Deterministic Rule Engine, State Machine Pipeline, Pre-Eligibility Live Check
Security & Auth	BCrypt Salted Hashing (10 rounds), Role-Based Access Control (RBAC), Zero-Bypass Login
Frontend Architecture	React 19, Vite 8.2, React Context API, Custom Hooks (usePlacementData), Glassmorphism
Quality Assurance	JUnit 5, Mockito Unit Testing, SpringBootTest Integration Suite (32/32 tests passed)